

JoeMoyo AutoTrader

The Complete Project Dossier

*How we designed, built, battle-tested, and packaged an automated paper-trading system
— the decisions, the mistakes, the fixes, and the evidence.*

Prepared 15 June 2026 · Live on Alpaca paper · A complete reference for reading or for NotebookLM audio.

Educational · Paper trading only · Not financial advice

JoeMoyo AutoTrader — The Complete Project Dossier

An end-to-end account of how we designed, built, battle-tested, and packaged an automated paper-trading system — the decisions, the mistakes, the fixes, the evidence, and exactly where things stand today.

Prepared: 15 June 2026 · **Status:** Live on Alpaca paper · **Mode:** Educational, paper trading only — not financial advice.

How to use this document

This dossier is written to be read straight through or fed into a tool like Google NotebookLM to generate an audio walkthrough. It deliberately favours full sentences and plain-English explanation over shorthand, so it narrates well. It moves from the big picture down into the detail: first what the system is and why it exists, then the strategy in plain terms, then the full chronological story of how it was built including every significant problem and how we solved it, then the hard evidence from backtesting, then the architecture, the risk controls, the memory layer, the honest limitations, where we stand right now, and finally the roadmap. A glossary at the end defines every term, which is especially useful if you are listening rather than reading.

A single thread runs through everything: this is a **paper-trading** system — it trades simulated money on a \$100,000 practice account. Nothing here is financial advice, and backtested results never guarantee future performance. The whole point of the exercise was to build the system properly, test it honestly, and keep a complete record so we always know where we stand.

Part 1 — Executive summary

Over the course of this project we built a fully automated stock-trading bot that runs by itself in the cloud, trades a simulated \$100,000 Alpaca paper account, and manages real risk with disciplined exits. It is not a black box and it is not a get-rich gimmick. It is a transparent, rules-based trend-following system with a serious supporting cast: a backtester, an out-of-sample validation suite, a live web dashboard, a self-writing trading journal, an AI coach that reviews its own performance, a durable memory in a database so nothing is ever lost, and a machine-checkable scorecard that tells us in plain language whether the strategy is passing or failing.

The trading idea itself is intentionally simple and well understood: a moving- average trend-following strategy. The bot buys strength and rides trends in large, liquid US stocks, and — critically — it protects capital with a trailing stop that locks in gains and cuts losers before they become disasters. We did not just assume this works. We tested it across four years of real market data, including the 2022 bear market, and we measured it honestly. The headline finding from that testing is that the strategy's edge is not about beating the market in a roaring bull run; it is about **losing far less when the market falls** while still capturing most of the upside. In the 2022 bear market the strategy lost about twenty-three percent where simply holding the same stocks lost about fifty-seven percent. That is the entire thesis in one sentence.

Two engines run side by side on the same account without interfering: a **swing engine** that trades daily price bars with a \$50,000 budget, and an **intraday engine** that trades five-minute bars with a \$10,000 budget and flattens everything before the closing bell so nothing is held overnight. Both run on automatic schedules via GitHub Actions, with no computer needed.

Everything the bot does is recorded — to a database, to CSV files, and to a daily log — and reviewed automatically each evening by an AI coach that surfaces recurring patterns and writes a plain-English performance note. A pass/fail scorecard grades the bot against six explicit criteria so we never have to guess whether it is working.

Finally, we packaged the entire build process — the code, the methodology, and the hard-won lessons — as a reusable Claude skill that can be shared or sold.

The system is live and trading. The one outstanding setup item is adding the Supabase database credentials as secrets, which switches on the durable-memory features. Everything else is running.

Part 2 — The trading strategy in plain English

Let us start with what the bot actually does in the market, explained as if to a friend with no trading background.

Every stock has a price that moves up and down all day. If you take the average closing price over the last twenty days, you get a smooth line that follows the recent trend — that is a "moving average." Take the average over fifty days and you get a slower, smoother line. When the faster twenty-day line is **above** the slower fifty-day line, the stock has been trending up recently; the short-term momentum is stronger than the long-term baseline. When the faster line drops **below** the slower one, momentum is fading.

Our bot buys when the fast line is above the slow line — it buys strength and rides uptrends — and it sells when momentum breaks. This is called an "SMA crossover" strategy, SMA standing for simple moving average. It is one of the oldest and most studied ideas in trend-following, precisely because it is transparent: you can look at a chart and see exactly why the bot did what it did. There is no mysterious artificial-intelligence prediction involved in the trading decision itself. The strategy is fully

deterministic, which means that given the same price data it will always make the same decision, and that makes it auditable and trustworthy.

We added one important refinement called **regime mode**. A naive crossover bot only acts on the exact day the two lines cross. If you switch it on during a trend that is already underway, it can sit on its hands for weeks or months waiting for the next crossing. Regime mode lets it recognise that a stock is *already* in an uptrend — fast line above slow line — and participate immediately, rather than waiting for a fresh cross. This is why, on its first real trading day, the bot was able to take positions straight away instead of doing nothing.

Now the most important part, and the part most amateur bots get catastrophically wrong: **how it sells**. A pure crossover strategy only exits when the slow averages finally cross back, which can take weeks. In that time a position could fall twenty or thirty percent while the bot just holds and watches. That is unacceptable risk management. So we added a **trailing stop**. A trailing stop works like this: it remembers the highest price the position has reached since you bought it, and if the price ever falls eight percent below that peak, it sells immediately. As the stock climbs, the stop climbs with it, locking in gains; if the stock reverses hard, the stop gets you out. The trailing stop is both the profit-protector and the crash-protector, and it fires regardless of what the moving averages are doing.

The bot trades a watchlist of ten large, liquid US stocks: the broad-market ETFs SPY and QQQ, plus Apple, Microsoft, Nvidia, Google, Amazon, Meta, Tesla, and AMD. It deploys at most \$50,000 of the \$100,000 account, roughly \$5,000 per position, across at most ten positions at a time. The rest of the account stays untouched as a buffer. There is also a smaller, faster intraday engine, which we will come to, that trades a completely separate set of five stocks so the two never collide.

Part 3 — The whole story, phase by phase

This is the heart of the dossier: the chronological journey, including the problems we hit and how we solved them. Each phase built on the last, and we shipped each one to a live branch and merged it before moving on.

Phase one — Getting onto the market

The foundation was connecting to Alpaca, a commission-free brokerage with an excellent API and, crucially, a free paper-trading mode that mirrors the real market with simulated money. We built a thin client to talk to Alpaca's REST API — fetching account information, current positions, open orders, and historical price bars, and placing orders. We deliberately kept the trading-strategy logic in pure, self-contained functions with no external dependencies, so the exact same code that decides a live trade can also be replayed over historical data in the backtester. This single decision — strategy logic as pure functions — paid off again and again, because it guaranteed that what we tested was what we traded.

A hard safety guardrail went in from day one: the automated loop refuses to run against anything other than a paper account. Even though Alpaca supports live trading, this version of the bot will not place real-money orders. That is a deliberate constraint, not a limitation we forgot to remove.

Phase two — Seeing it: the dashboard

A trading system you cannot see is a trading system you cannot trust. We built a live web dashboard using Streamlit that shows account equity, cash, buying power, open positions with their profit and loss, recent orders, and candlestick charts for each watchlist symbol with the moving averages drawn on top and the current signal labelled. The dashboard is the window into the bot, and it works on a phone, which matters because the whole point was to be able to glance at it from anywhere.

We deployed it free on Streamlit Community Cloud. This is where we hit our first friction. Streamlit Cloud could not initially see the repository because it was private, and the connection only covers repositories you explicitly authorise. There was a back-and-forth getting the GitHub authorisation right, and a separate issue where the Alpaca keys had to be provided to Streamlit as "secrets" in a specific TOML format — and the format is fussy, rejecting smart quotes and malformed lines. We got it connected and live, and the user confirmed seeing the dashboard.

Phase three — Running itself: cloud automation

A bot that only runs when your laptop is open is not really automated. We set up GitHub Actions — free scheduled jobs that run in GitHub's cloud — to execute one trading cycle every thirty minutes during US market hours. This is where we hit the single most time-consuming gotcha of the entire project.

GitHub only runs scheduled workflows from a repository's default branch. Our work was on a feature branch, so the schedule simply never fired. The Actions tab appeared empty and it looked broken. The fix was to merge the work into the default branch. This is a genuinely non-obvious rule that wastes a lot of people's time, and it is now documented prominently in our lessons file. We also learned that GitHub's cron scheduling is approximate — runs can be delayed ten or fifteen minutes and the scheduler takes a while to warm up — which is fine for a swing bot on daily bars but matters for the intraday engine later.

Phase four — Discipline: budgets and the first backtest

We added an explicit budget cap so the bot can never deploy more than a set amount — protecting the rest of the account — and we built the first version of the backtester. The backtester replays the exact same signal and sizing rules over historical price data and reports the strategy's return versus simply buying and holding, its maximum drawdown, its Sharpe ratio, win rate, and every individual trade. From this point on, no change went live without being backtested first.

Phase five — The credential incident

This is the most important safety story in the project, and we are documenting it honestly because it matters. To activate the automation, the Alpaca keys had to be added as GitHub repository secrets. There are two keys: a short, non-secret key ID, and a long secret key. The whole multi-line block — both keys plus a setting — was accidentally pasted into the single secret field meant only for the secret key.

When the bot ran, it tried to send that malformed blob as an API header, which both failed and, worse, **echoed the secret key into the run log of what was at that point a public repository**. We caught it, deleted those run logs immediately, and recommended rotating the keys as a precaution. Because this is a paper-trading key with no access to real money, the actual risk was low, but the principle is serious. We then added a permanent defence in the code: a sanitiser that detects when a whole settings block has been pasted into one variable and extracts just the correct value, so this class of mistake can never break the bot or leak again. The lesson — never paste a whole key-equals-value block into a single secret, and remember that a public repo's logs are public — is now front and centre in our documentation.

Phase six — Trading on day one: regime mode

As described earlier, we added regime mode so the bot engages with trends already in progress rather than waiting for a fresh crossover. We verified through backtesting that in a steady uptrend the old crossover-only logic would never trade, while regime mode correctly entered and captured the move. This is what made the first live trading session productive.

Phase seven — Scaling up: the \$50,000 configuration

We scaled the bot to a \$50,000 budget across a ten-stock watchlist, with roughly \$5,000 per position and a maximum of ten positions. We added "participation gates" — a check that refuses to trade any stock that is not liquid enough, requiring meaningful average volume and daily range — so the bot will never trade a thin, untradeable name. We also added "top-up" logic so that when the budget was raised, the bot would bring its existing under-sized positions up to the new target size rather than leaving cash idle.

Importantly, we tested two filters suggested by professional trading content — a volume-confirmation filter and a broad-market filter — and the backtest showed they actually *reduced* returns and risk-adjusted performance for this particular strategy on these particular stocks. So we built them but left them switched off by default, available for experimentation. This is a recurring theme: we let the data overrule plausible-sounding ideas.

Phase eight — Better data: the SIP feed

When the user upgraded their Alpaca data subscription, we switched the system from the free IEX data feed to the full consolidated SIP feed. This was a bigger deal than it sounds. The free IEX feed only reports the volume that trades on one small exchange — for SPY it showed about one million shares where the true whole-market figure was over twenty-six million. Any logic that depends on volume was

effectively blind on the free feed. We switched to SIP for accurate prices and real volume, with an automatic fallback to IEX if the subscription ever lapses, so the bot can never be left without data.

Phase nine — The part most bots skip: risk-managed exits

This was a turning point. We added the trailing stop, and rather than guessing the settings we ran a proper bake-off across four years of data on all ten stocks. The results were decisive and, frankly, surprising. The original idea was a balanced package: a five-percent stop loss, a fifteen-percent take-profit target, and an eight-percent trailing stop. The data showed that the take-profit target actually *hurt* — it capped the winners in a market where the big stocks ran far beyond fifteen percent, cutting total return from about seventy-five percent down to about forty-eight percent. We then tried adding a fixed catastrophe stop, and that hurt too — it ejected positions on violent dips that subsequently recovered, missing the rebound, dragging return down to about fifty-eight percent.

The clean winner was the **trailing stop alone**: about seventy-five percent return, the best risk-adjusted return of any configuration, and the lowest drawdown. The trailing stop is its own crash protection; bolting fixed stops and targets onto it only caused premature exits. So the live configuration became a single eight-percent trailing stop, no take-profit, no fixed stop. This beat the original signal-only bot on return, on Sharpe ratio, and on drawdown simultaneously. The lesson — do not assume, backtest your own universe — is one of the most valuable in the whole project.

Phase ten — Day trading: the intraday engine

The user wanted day trades, so we built a second, faster engine. It trades five-minute bars with faster nine and twenty-period moving averages, a tighter two-percent trailing stop, a daily-loss circuit breaker that stops new trades once losses hit \$500 in a day, and — most importantly — an end-of-day flatten that closes every intraday position about five minutes before the closing bell so nothing is ever held overnight.

The critical engineering challenge was that this second engine shares the same Alpaca account as the swing engine. Without care, the intraday bot would see the swing bot's positions, mis-calculate its own budget against them, and even flatten them at the close. We solved this by scoping everything — budget, position counts, and the end-of-day flatten — to each engine's own symbols, and by giving the intraday engine a completely separate watchlist of five different stocks. With disjoint watchlists and per-engine scoping, the two bots coexist on one account safely. We were honest in the documentation that because GitHub's scheduler is coarse, this is minutes-to-hours intraday trading, not true high-frequency scalping, with the end-of-day flatten as the guaranteed backstop.

Phase eleven — Self-awareness: the journal and AI coach

We made the system log and review itself. A journal module pairs up the bot's buy and sell orders into completed round-trip trades using first-in-first-out matching, and computes statistics: win rate, profit factor, profit and loss broken down by stock and by exit reason, average holding time, and a daily profit-

and- loss calendar. On top of that sits an AI coach — this is where Claude is used, and used appropriately. The coach does not predict the market or pick trades. It *reviews* what already happened, the way a desk coach reviews the day's tape, and writes a plain-English performance note plus a list of recurring "tendencies" — patterns worth watching. It runs automatically every evening after the close via a scheduled job, and it feeds its previous observations back into each new review so the analysis compounds over time. If there is no AI key configured, it gracefully falls back to a deterministic statistics-only summary, so the report always generates.

The dashboard gained a whole journal-and-coach section with tabs for performance, the round-trip trade history, the coach's notes and tendencies, and a manual journal where the user can log their own discretionary observations and grades.

Phase twelve — Never forgetting: Supabase durable memory

A cloud bot runs in disposable containers, and Streamlit Cloud wipes its filesystem on restart, so anything written at runtime can vanish. To give the system a permanent memory, we connected it to Supabase, a hosted Postgres database. The user already had an "Investment" database there containing their real personal UK investment records — holdings, ISA allowances, dividend history, thousands of snapshots. Protecting that real data was paramount, so we created a completely separate set of tables, clearly prefixed, that hold only the paper bot's test data and never touch the real records.

We hit one more instructive snag here. We initially created a dedicated database schema, but Supabase's web API does not expose custom schemas by default, which would have required a manual configuration step we could not verify. So we moved the tables into the standard public area under a clear "bot" prefix — same isolation, but guaranteed to work over the API with no extra setup. Every trade, every daily equity snapshot, the round-trip ledger, the coach's notes and tendencies, the pattern statistics, and the manual journal now persist durably. The whole storage layer is best-effort: if the database is ever unreachable, the bot keeps trading and logs to files instead — logging can never break trading.

Phase thirteen — Proving it: validation and the scorecard

Backtesting tells you *if* something worked in the past; it does not tell you whether the result was robust or just a lucky fit. So we built a proper validation battery. First we fixed the data layer to paginate properly — the price-history endpoint caps at a thousand bars per request, and without following the continuation tokens you silently get a truncated window, which would quietly corrupt any long backtest. With that fixed we could fetch years of daily data and weeks of minute data whole.

The validation suite does three things. **Walk-forward analysis** runs the strategy over consecutive ninety-day out-of-sample windows and checks consistency window after window. **Regime testing** runs it separately over the 2022 bear market and the 2023-to-2024 bull market to see how it behaves in each. **Parameter robustness** sweeps across different moving-average lengths and trailing-stop widths to confirm the live settings sit on a broad plateau of similar results — a sign of a real edge — rather than a lone fragile spike, which would signal overfitting.

Then we wrote the strategy specification — the bot's contract — and turned its pass/fail criteria into code. The scorecard evaluates six measurable criteria and returns a verdict in plain language: PASS, WATCH, FAIL, or IN PROGRESS. Crucially, it measures the bot's own profit and loss against its budget, not against the whole account, because the idle cash would otherwise dilute every number and make the drawdown look artificially small.

Phase fourteen — Packaging it: the shareable skill

Finally, the user asked to capture the entire process as something they could share or sell. We packaged everything as a reusable Claude skill: a guided, nine-phase build walkthrough, deep-dive reference documents for each phase, a lessons-learned file capturing every pitfall above, and a complete set of genericised, credential-free code templates so the whole bot can be scaffolded fresh for a new user. It includes a packaging script that builds a distributable zip with a built-in secret-scan guard, plus licence and disclaimer files for selling it responsibly.

Part 4 — The evidence: what the testing actually showed

It is worth laying out the real numbers, because honest evidence is the whole point of battle-testing. All figures below come from backtests over roughly four years of real market data, 2022 through 2026, on the ten-stock swing watchlist, with a \$50,000 budget and regime mode, unless stated otherwise. Remember throughout: these are historical simulations, they exclude slippage and fees, and they do not guarantee anything about the future.

The exit bake-off

Configuration	Return	Sharpe	Max drawdown
Signal-only (no risk exit)	~70.6%	0.97	16.2%
Stop 5% + take-profit 15% + trail 8%	~48%	0.86	12.7%
Trailing stop 8% only (chosen)	~74.9%	1.19	14.2%
Trailing stop 15% only	~80.5%	1.14	16.2%
Catastrophe stop 12% + trail 8%	~58%	0.92	14.7%

The trailing-stop-only configuration beat the original bot on all three measures at once — higher return, better risk-adjusted return, and lower drawdown. The take-profit and fixed-stop variants were measurably worse. This is why the live bot runs a single eight-percent trailing stop.

Walk-forward (out-of-sample consistency)

Across seventeen consecutive ninety-day out-of-sample windows, eleven were positive, with an average of roughly plus-six-point-eight percent and a range from about minus-ten percent in the worst window to plus-thirty-four percent in the best. The pattern that matters: in the losing windows, the strategy lost far less than simply holding the stocks. In the very worst window it was down about ten percent while buy-and-hold was down about forty percent.

Behaviour across market regimes

Period	Strategy	Buy & hold
2022 bear market	−23%	−57%
2023–2024 bull market	+61%	+132%
Recent (2025–2026)	+30%	+55%

This table *is* the thesis. In the bear market the strategy cut the loss by more than half — that is the trailing stop and trend-following doing exactly their job. In the bull markets it trailed raw buy-and-hold, which is the expected and accepted trade-off for a system whose priority is protecting capital. A trend-follower is not trying to beat a rocket; it is trying to avoid the crashes.

Parameter robustness

The sweep across moving-average lengths and trailing widths produced a broad cluster of similar results around the live configuration, confirming the edge is not an overfit fluke. One finding of note: a slightly faster moving-average pair scored marginally better on risk-adjusted return. We deliberately did **not** rush to adopt it. The discipline is that a parameter change only goes live after walk-forward analysis confirms it out-of-sample, so it is flagged as a candidate for a controlled future experiment, not a knee-jerk change.

The first live trades

On the bot's first proper live session, once everything was wired correctly, a single cycle bought into all eligible names — topping up the five starter positions to target size and opening new positions in Google, Amazon, Tesla, and AMD — deploying the budget exactly as designed. It correctly declined to buy Meta because Meta's short-term trend was below its long-term trend at that moment. That single decision — buying the trends, skipping the one that did not qualify — was the clearest possible proof the discipline was working as specified.

Part 5 — How the whole machine fits together

It helps to picture the system as a set of cooperating parts, each with one job.

At the base is the **Alpaca client**, the only part that talks to the outside market — fetching prices and account data and placing orders, on the accurate SIP data feed with an automatic fallback. Above it sits the **strategy core**, a set of pure mathematical functions that turn price bars into a buy, sell, or hold signal and decide position size and exit triggers. Because these are pure functions, the **backtester** and the **validator** can call the exact same logic over historical data — what we test is genuinely what we trade.

The **auto-trader loop** is the conductor. Each cycle it checks whether the market is open, refreshes the account and positions, and for each stock pulls the latest bars, checks the risk-exit rules first, then the signal, and proposes buys or sells within the budget and position caps — placing the orders, or merely logging them in dry-run mode, and recording every decision.

Off to the side, never blocking the trading, are the supporting systems. The **dashboard** renders everything live in a browser. The **journal** pairs trades into round trips and computes statistics. The **coach** has Claude narrate the results and flag tendencies. **Supabase** stores all of it durably. The **scorecard** grades the whole thing against the strategy contract. And it is all driven from a single command-line interface with subcommands for trading, backtesting, validating, coaching, and scoring.

A guiding principle throughout is that the trading itself is deterministic and free to run — there is no AI call in the trading decision, which keeps it auditable and cheap to run continuously. Artificial intelligence is used only to *review* performance after the fact and, optionally, as a conservative second opinion that can veto a risky trade — never to predict prices.

Part 6 — Risk management and the scorecard

Risk control is not an afterthought in this system; it is the centrepiece. There are layers. The budget cap limits total capital at risk. The per-position size limit prevents any one stock from dominating. The participation gates keep the bot out of illiquid names. The trailing stop protects every individual position. The intraday engine adds a daily-loss circuit breaker and a guaranteed end-of-day flatten. And there is a drawdown brake in the operating plan: if the bot's equity falls more than ten percent from its peak, position sizes are halved until the next review.

On top of all that sits the scorecard, which turns "is it working?" from a gut-feel question into a measured verdict. It checks six things: that the bot is profitable on its budget; that its risk-adjusted return, the Sharpe ratio, clears a sensible bar; that its maximum drawdown stays within fifteen percent; that its drawdown is no worse than the market's over the same period — the downside-edge test that proves the core thesis; that its profit factor, the ratio of gross wins to gross losses, is healthy; and that the risk controls are demonstrably firing. There are hard-fail tripwires — a drawdown beyond twenty-five percent, or any single trade losing more than twenty-five percent — that force a FAIL regardless. And

there is a data gate: until there are at least twenty trading days and ten completed trades, the verdict is simply IN PROGRESS, because judging too early is just noise. Today, before the campaign has really begun, the scorecard correctly reads IN PROGRESS.

The discipline that ties it together: the scorecard is the referee, and the validator is how we earn the right to change a rule. One change at a time, always confirmed out-of-sample before it goes live.

Part 7 — Durable memory and the self-improving loop

The combination of the journal, the coach, and the Supabase memory creates something more than logging — it creates a feedback loop. Every trade and a daily snapshot of the bot's own profit and loss against its budget are written to the database. Each evening the coach rebuilds the complete round-trip ledger, computes the statistics, and writes its note and tendencies — and it reads its previous tendencies back in, so instead of starting fresh each day it builds a cumulative understanding of how the bot behaves. Over a ninety-day campaign this becomes a genuine record of what is working, which stocks and which exit reasons drive the profit and loss, and where the weaknesses are. The manual journal lets the human add discretionary observations alongside the bot's own, and because it all lives in the database, none of it is lost when a container restarts.

This is the honest, useful way to use AI in trading: not as a fortune-teller, but as a tireless analyst and record-keeper that turns raw history into insight.

Part 8 — The lessons, collected

Several hard-won lessons recur, and they are worth stating plainly because they are the most transferable value in the whole project.

On strategy: take-profit targets and fixed stops both hurt this trend-following strategy; a trailing stop alone won on every measure. Never assume — backtest your own universe. A fresh crossover bot can sit idle for months without regime mode.

On data: the free IEX feed shows only a few percent of real volume, making any volume logic meaningless; use the full SIP feed. And always paginate historical data, or you silently get a truncated, misleading window.

On automation: scheduled jobs only run from the default branch — this is the classic trap. The scheduler is also approximate, so design for delay.

On secrets: never paste a whole key-equals-value block into a single secret field; it both breaks things and can leak into logs. A public repository's logs are public. If a secret ever appears, rotate it and delete the logs immediately.

On persistence: hosted dashboards have throwaway filesystems, so persist to a database; and custom database schemas may not be reachable over the web API, so a clear table prefix in the standard area is the pragmatic choice.

On process: the backtester must call the same code the live bot uses, or you are testing a different system. Do not chase the backtest peak; confirm changes out-of-sample. And judge a trend-follower on risk-adjusted return and downside protection, not on raw return versus the market.

Part 9 — Where we stand today

As of today the system is built, tested, merged, and live. The swing engine is configured for \$50,000 and runs every thirty minutes during market hours. The intraday engine is configured for \$10,000 on a separate watchlist with an end-of-day flatten. The dashboard is deployed and shows the live account plus the journal, coach, and scorecard. The backtester, validator, journal, coach, and scorecard all work and have been verified. The whole build has been merged through a series of clean pull requests, and the existing system was never disturbed when we added new capabilities.

There is one outstanding setup item that is the user's to complete: adding the Supabase database credentials — the project URL and the service key — as secrets in both GitHub and Streamlit. Until that is done, the bot still trades and still logs to files, but the durable-memory features and the parts of the scorecard that depend on the stored equity history stay dormant. Optionally, adding an Anthropic key as a secret switches the coach from deterministic statistics to a written narrative. Neither is required for trading; both enrich the memory and review layers.

It is also worth recording, for full transparency, that during the project the repository was made public at one point, which is what made the brief secret-leak incident sensitive; the recommendation to rotate the paper keys as a precaution stands, and is low-urgency because they are paper-only keys.

The scorecard today reads IN PROGRESS — day zero, no completed trades — which is exactly correct, because the ninety-day evaluation has only just begun and a verdict before there is data would be meaningless.

Part 10 — The road ahead: the ninety-day plan

The plan from here is a disciplined ninety-day battle-test, both engines running, reviewed weekly. The first two weeks are deploy-and-observe: let it run, watch the dashboard daily, change nothing, and confirm the database fills and the evening coach note appears. Weeks three and four are the first real read of the data, with a weekly coach review of win rate, profit factor, and profit and loss by stock and exit reason — and, at the end of week four, a controlled walk-forward test of the faster moving-average candidate that the parameter sweep flagged, adopted only if it genuinely beats the live config out-of-

sample. Weeks five through eight are for stress and refinement, one isolated, backtested change at a time, watching how the trailing stop performs through any market drawdown. Weeks nine through twelve are the verdict: a full review from the stored history against the scorecard — total return versus the budget, versus the market, the drawdown, the win rate, the profit factor, and the swing-versus-intraday contribution — and a decision to keep, adopt a validated improvement, rescale, or retire an engine, with the next period's plan written from the data rather than from how the months felt.

Throughout, the standing rules hold: paper only, exits never filtered, one change at a time and only after validation, the drawdown brake armed, and everything logged so we never trade blind.

Part 11 — The reusable skill

Beyond the running bot, the project produced a second, durable asset: a packaged Claude skill that captures the entire methodology. It contains the guided nine-phase build, a deep-dive reference for each phase, the complete lessons-learned file, and a full set of genericised, credential-free code templates so the whole system can be rebuilt for anyone from scratch. It ships with a packaging script that produces a clean distributable archive — with an automatic scan that refuses to package if any credential slipped in — and with licence and disclaimer files so it can be shared or sold responsibly. In effect, the project's knowledge is now reproducible and transferable, not locked in one person's head.

Part 12 — Honest limitations and risks

In the interest of always knowing exactly where we stand, here are the candid caveats. This is paper trading; simulated fills are kinder than real ones, and live trading introduces slippage, partial fills, and emotional factors the simulation does not capture. Backtests have hindsight built in and exclude fees and slippage; a strategy that backtests beautifully can still lose money live. Trend-following structurally lags in strong bull markets — it will underperform a simple buy-and-hold in a rip, and that is by design, not a defect. The strategy was validated on large, liquid US mega-caps; it should not be assumed to work on small, illiquid, or highly volatile names. GitHub's scheduler is approximate, so the intraday engine is not true scalping. And the whole thing depends on external services — Alpaca, GitHub, Streamlit, Supabase — any of which can have an outage, which is precisely why logging is best-effort and never allowed to block trading.

None of this is financial advice. It is an engineering and educational project, run on simulated money, with honest measurement as its guiding value.

Glossary

Alpaca — the commission-free brokerage whose API the bot uses; it offers a free paper-trading mode with simulated money.

Paper trading — trading with simulated money on real market data; no real funds are at risk.

SMA (simple moving average) — the average closing price over a number of days; a smoothed line that follows the trend.

SMA crossover — buying when a faster average rises above a slower one and selling when it falls below; the core signal.

Regime mode — entering a stock already in an uptrend rather than waiting for a fresh crossover.

Trailing stop — an exit that follows the highest price reached and sells if price falls a set percentage below that peak; protects gains and limits losses.

Drawdown — the percentage drop from a peak to a subsequent trough; the key measure of pain and risk.

Sharpe ratio — return divided by volatility; a measure of risk-adjusted return, where higher is better.

Profit factor — gross profits divided by gross losses; above one means the winners outweigh the losers.

Backtest — replaying the strategy over historical data to see how it would have performed.

Walk-forward analysis — testing over consecutive out-of-sample windows to check the edge is consistent, not a one-off.

Overfitting — tuning a strategy so tightly to past data that it fails on new data; the enemy of robust testing.

Buy-and-hold — simply buying and keeping the stocks; the benchmark the strategy is compared against.

SIP feed — the consolidated market data feed covering all exchanges, giving true prices and volume; the free IEX feed covers only a fraction.

GitHub Actions — GitHub's cloud automation that runs the bot on a schedule.

Streamlit — the framework powering the live web dashboard.

Supabase — the hosted Postgres database providing durable memory.

Round-trip trade — a completed buy-then-sell pair, the unit of the journal's profit-and-loss accounting.

The coach — the AI layer that reviews completed trades and writes a performance note plus recurring tendencies; it reviews, it does not predict.

Scorecard — the automated pass/fail grade against the strategy's six criteria.

Swing engine — the daily-bars bot with the \$50,000 budget.

Intraday engine — the five-minute-bars bot with the \$10,000 budget that flattens before the close.

Closing

We set out to build a real, honest, automated trading system and to keep a complete record of where we stand. We have a live, self-running bot with serious risk controls, an evidence base from four years of testing, a memory that compounds, a referee that grades it, and a packaged skill that makes the whole thing reproducible. The numbers are promising in exactly the way a sound trend-follower's numbers should be — strong downside protection, honest upside — and we have been candid about every limitation and every mistake along the way. The next chapter is the ninety-day campaign, run with discipline and measured against a scorecard that cannot be fooled.

End of dossier. Educational, paper trading only — not financial advice.